

Trees: Binary Trees and Binary Search Trees - Complete Guide

Introduction to Trees

A **tree** is a non-linear hierarchical data structure consisting of nodes connected by edges, where each node contains data and references to its child nodes^{[262][271]}. Unlike linear data structures (arrays, linked lists, stacks, queues), trees represent hierarchical relationships and enable efficient searching, insertion, and deletion operations^{[259][269]}.

Why Trees? The Need for Hierarchical Data Structures

Linear data structures have limitations when representing hierarchical relationships^[^271]:

Problems with Linear Structures:

- Cannot efficiently represent parent-child relationships
- Search operations in unsorted arrays: $O(n)$
- Maintaining sorted order in arrays requires expensive shifts
- Linked lists require sequential traversal

Real-World Hierarchical Data:

- File systems (directories and subdirectories)
- Organization charts (employees and managers)
- HTML DOM (nested HTML elements)
- Expression parsing (mathematical expressions)
- Database indexing (B-trees, B+ trees)

Trees solve these problems by providing $O(\log n)$ operations in balanced cases while naturally representing hierarchical data^{[263][269]}.

Tree Terminology

Understanding tree terminology is essential for mastering tree operations^{[256][262]}:

Node: The fundamental unit containing data and references to children

```
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};
```

Root: The topmost node with no parent (entry point to tree)^{[256][259]}

Parent: A node that has one or more child nodes

Child: A node directly connected below another node

Leaf (External Node): A node with no children (terminal nodes)^[^256]

Internal Node: A node with at least one child^[^256]

Edge: The connection between two nodes (parent-child link)

Subtree: A tree formed by a node and all its descendants

Height of Node: Longest path from that node to a leaf^[^256]

- Height of leaf node = 0
- Height of tree = height of root

Depth of Node: Number of edges from root to that node^[^256]

- Depth of root = 0

Level: All nodes at the same depth (root is level 0)^[^259]

Degree of Node: Number of children that node has

Sibling: Nodes that share the same parent

Ancestor: All nodes on path from root to that node

Descendant: All nodes in subtree rooted at that node

Tree Properties

General Tree Properties:

- Maximum nodes at level i : 2^i (for binary tree)
- Maximum nodes in tree of height h : $2^{(h+1)} - 1$
- Minimum height for n nodes: $\lceil \log_2(n+1) \rceil - 1$
- Number of edges = Number of nodes - 1

Binary Trees

A **binary tree** is a tree data structure where each node has **at most two children**, referred to as the **left child** and **right child**^{[253][262][^265]}.

Binary Tree Definition

Formal Definition:

A binary tree is a finite set of nodes that is either:

- Empty, or
- Consists of a root node and two disjoint binary trees called left subtree and right subtree

Key Characteristic: Maximum 2 children per node (0, 1, or 2)^{[253][262]}

Node Structure in C

```
struct Node {  
    int data;  
    struct Node* left;  
    struct Node* right;  
};
```

Components:

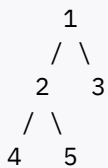
- `data`: Stores the value
- `left`: Pointer to left child (NULL if no left child)
- `right`: Pointer to right child (NULL if no right child)

Types of Binary Trees

Understanding different binary tree classifications is crucial for algorithm selection^{[253][259][262][268]}:

1. Full Binary Tree (Strict Binary Tree)

Definition: Every node has either 0 or 2 children (no node has exactly 1 child)^{[253][262][268]}

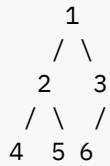


Properties:

- Number of leaf nodes = Number of internal nodes + 1
- Total nodes = 2 × leaf nodes - 1
- Used in expression trees

2. Complete Binary Tree

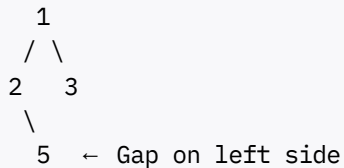
Definition: All levels are completely filled except possibly the last level, which is filled from left to right



Properties:

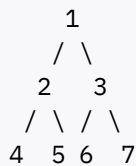
- Height = $\lfloor \log_2(n) \rfloor$
- Used in heap data structures
- Efficient array representation possible
- No gaps in array representation

Not Complete:



3. Perfect Binary Tree

Definition: All internal nodes have exactly 2 children, and all leaf nodes are at the same level

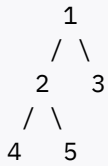


Properties:

- Most restrictive type
- Number of nodes = $2^{h+1} - 1$
- Height h tree has 2^h leaf nodes
- Perfectly balanced
- All levels completely filled

4. Balanced Binary Tree

Definition: For every node, the height difference between left and right subtrees is at most 1^[259]_[262]
^[^268]

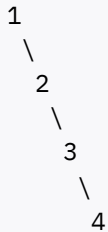


Properties:

- Height remains $O(\log n)$
- Ensures efficient operations
- AVL trees and Red-Black trees maintain balance
- Height difference ≤ 1 for all nodes

5. Degenerate (Pathological) Tree

Definition: Each internal node has only one child^[^262]



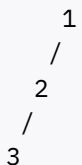
Properties:

- Performance degrades to $O(n)$ like linked list
- Height = $n - 1$ (worst case)
- Left-skewed or right-skewed

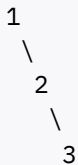
6. Skewed Binary Tree

Definition: A degenerate tree where all nodes lean to one side^[^262]

Left-Skewed:



Right-Skewed:



Binary Tree vs. General Tree

Aspect	Binary Tree	General Tree
Max Children	2	Unlimited
Child Identity	Left/Right distinction	Order matters
Empty Subtree	Can have empty left or right	N/A
Applications	BST, heaps, expression trees	File systems, DOM

Binary Search Tree (BST)

A **Binary Search Tree** is a binary tree with an additional ordering property that enables efficient searching, insertion, and deletion[257][263][^269].

BST Property (The Fundamental Rule)

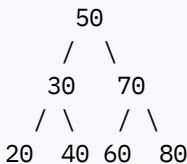
For every node in a BST[257][260][^263][269]:

- Left Subtree Property:** All values in left subtree < node's value
- Right Subtree Property:** All values in right subtree > node's value
- Recursive Property:** Both left and right subtrees are also BSTs

Mathematical Notation:

left_subtree(keys) < node(key) < right_subtree(keys)

BST Example

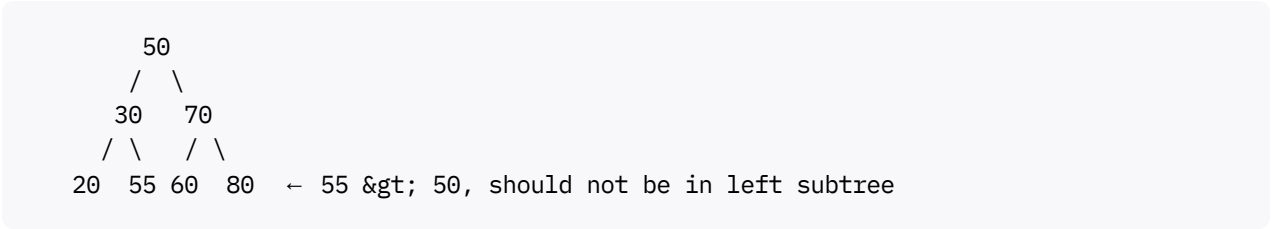


Verification:

- At 50: left subtree (20,30,40) < 50 < right subtree (60,70,80) ✓

- At 30: left (20) < 30 < right (40) ✓
- At 70: left (60) < 70 < right (80) ✓

Not a BST:



Why BST is Powerful

Comparison with Other Structures:

Operation	Array (unsorted)	Array (sorted)	Linked List	BST (balanced)
Search	O(n)	O(log n)	O(n)	O(log n)
Insert	O(1)	O(n)	O(1)	O(log n)
Delete	O(n)	O(n)	O(n)	O(log n)
Min/Max	O(n)	O(1)	O(n)	O(log n)
Sorted Order	O(n log n)	O(n)	O(n)	O(n)

BST Advantages:

- Combines benefits of arrays (fast search) and linked lists (fast modification)
- Maintains sorted order automatically
- Average O(log n) for all operations
- In-order traversal gives sorted sequence

Tree Traversal Algorithms

Tree traversal is the process of visiting each node in the tree exactly once in a specific order^{[255][261][264][267][^270]}. Unlike linear structures, trees offer multiple meaningful traversal orders.

Why Multiple Traversal Methods?

Different traversals serve different purposes^{[258][267]}:

- **Inorder:** Get sorted sequence from BST
- **Preorder:** Create copy of tree, prefix expressions
- **Postorder:** Delete tree, postfix expressions, directory size calculation
- **Level-order:** BFS, shortest path in unweighted tree

Depth-First Traversals (DFS)

Depth-First Search explores as far as possible along each branch before backtracking^{[254][267][^270]}.

1. Inorder Traversal (Left-Root-Right)

Order: Left subtree → Root → Right subtree^{[255][261][^264][267]}

Algorithm:

```
Inorder(node):  
    if node is NULL:  
        return  
    Inorder(node->left)  
    visit(node)  
    Inorder(node->right)
```

Example:



Inorder Result: 1, 2, 3, 4, 5, 6, 7 (sorted order!)

Key Property: For BST, inorder traversal produces values in **ascending sorted order**^{[267][270]}

Applications:

- Get sorted elements from BST
- Evaluate expressions in expression trees
- Validate if tree is BST

2. Preorder Traversal (Root-Left-Right)

Order: Root → Left subtree → Right subtree^{[255][261][^264][267]}

Algorithm:

```
Preorder(node):  
    if node is NULL:  
        return  
    visit(node)  
    Preorder(node->left)  
    Preorder(node->right)
```

Example:



Preorder Result: 4, 2, 1, 3, 6, 5, 7

Applications:

- Create copy of tree (visit parent before children)
- Serialize tree structure
- Prefix expression evaluation
- Tree structure printing

3. Postorder Traversal (Left-Right-Root)

Order: Left subtree → Right subtree → Root^[255][261]^[264][267]

Algorithm:

```

Postorder(node):
    if node is NULL:
        return
    Postorder(node->left)
    Postorder(node->right)
    visit(node)

```

Example:



Postorder Result: 1, 3, 2, 5, 7, 6, 4

Applications:

- Delete tree (delete children before parent)
- Postfix expression evaluation
- Calculate directory size (sum children first)
- Memory deallocation

Breadth-First Traversal (BFS)

Level-Order Traversal

Order: Visit nodes level by level, left to right

Algorithm (using queue):

```
LevelOrder(root):  
    if root is NULL:  
        return  
    queue.enqueue(root)  
    while queue is not empty:  
        node = queue.dequeue()  
        visit(node)  
        if node->left exists:  
            queue.enqueue(node->left)  
        if node->right exists:  
            queue.enqueue(node->right)
```

Example:



Level-Order Result: 4, 2, 6, 1, 3, 5, 7

Applications:

- Print tree level by level
- Find shortest path in unweighted tree
- Check if tree is complete
- Serialize tree with level information

Traversal Complexity

All traversal methods:

- **Time Complexity:** $O(n)$ - visit each node once
- **Space Complexity:**
 - Recursive: $O(h)$ where h = height (call stack)
 - Iterative (Level-order): $O(w)$ where w = maximum width

Binary Tree Implementation in C

Basic Node Structure and Tree Creation

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

// Binary tree node structure
typedef struct Node {
    int data;
    struct Node* left;
    struct Node* right;
} Node;

// Create a new node
Node* createNode(int data) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    if (!newNode) {
        printf("Memory allocation failed!\n");
        return NULL;
    }
    newNode->data = data;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}

// Example: Create a sample binary tree
Node* createSampleTree() {
    /*
    * Creating tree:
    *       1
    *      / \
    *     2  3
    *    / \  \
    *   4  5  6
    */
    Node* root = createNode(1);
    root->left = createNode(2);
    root->right = createNode(3);
    root->left->left = createNode(4);
    root->left->right = createNode(5);
    root->right->right = createNode(6);

    return root;
}

int main() {
    printf("=== Binary Tree Creation Demo ===\n");
    Node* root = createSampleTree();
    printf("Tree created successfully!\n");
    printf("Root: %d\n", root->data);
    printf("Root->left: %d\n", root->left->data);
    printf("Root->right: %d\n", root->right->data);
}
```

```
    return 0;
}
```

Tree Traversal Implementations

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node* left;
    struct Node* right;
} Node;

Node* createNode(int data) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = data;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}

// Inorder Traversal: Left -> Root -> Right
void inorderTraversal(Node* root) {
    if (root == NULL) return;

    inorderTraversal(root->left);
    printf("%d ", root->data);
    inorderTraversal(root->right);
}

// Preorder Traversal: Root -> Left -> Right
void preorderTraversal(Node* root) {
    if (root == NULL) return;

    printf("%d ", root->data);
    preorderTraversal(root->left);
    preorderTraversal(root->right);
}

// Postorder Traversal: Left -> Right -> Root
void postorderTraversal(Node* root) {
    if (root == NULL) return;

    postorderTraversal(root->left);
    postorderTraversal(root->right);
    printf("%d ", root->data);
}

// Level Order Traversal (BFS) using queue
typedef struct QueueNode {
    Node* treeNode;
    struct QueueNode* next;
} QueueNode;
```

```

typedef struct {
    QueueNode* front;
    QueueNode* rear;
} Queue;

Queue* createQueue() {
    Queue* q = (Queue*)malloc(sizeof(Queue));
    q->front = q->rear = NULL;
    return q;
}

void enqueue(Queue* q, Node* node) {
    QueueNode* temp = (QueueNode*)malloc(sizeof(QueueNode));
    temp->treeNode = node;
    temp->next = NULL;

    if (q->rear == NULL) {
        q->front = q->rear = temp;
        return;
    }
    q->rear->next = temp;
    q->rear = temp;
}

Node* dequeue(Queue* q) {
    if (q->front == NULL) return NULL;

    QueueNode* temp = q->front;
    Node* node = temp->treeNode;
    q->front = q->front->next;

    if (q->front == NULL) q->rear = NULL;

    free(temp);
    return node;
}

bool isEmpty(Queue* q) {
    return q->front == NULL;
}

void levelOrderTraversal(Node* root) {
    if (root == NULL) return;

    Queue* q = createQueue();
    enqueue(q, root);

    while (!isEmpty(q)) {
        Node* current = dequeue(q);
        printf("%d ", current->data);

        if (current->left) enqueue(q, current->left);
        if (current->right) enqueue(q, current->right);
    }

    free(q);
}

```

```

}

// Demonstration
int main() {
    // Create tree:
    //      4
    //     / \
    //    2   6
    //   / \ / \
    //  1  3 5  7

    Node* root = createNode(4);
    root->left = createNode(2);
    root->right = createNode(6);
    root->left->left = createNode(1);
    root->left->right = createNode(3);
    root->right->left = createNode(5);
    root->right->right = createNode(7);

    printf("=== Tree Traversal Demonstrations ===\n\n");

    printf("Inorder Traversal (Left-Root-Right): ");
    inorderTraversal(root);
    printf("\n");

    printf("Preorder Traversal (Root-Left-Right): ");
    preorderTraversal(root);
    printf("\n");

    printf("Postorder Traversal (Left-Right-Root): ");
    postorderTraversal(root);
    printf("\n");

    printf("Level Order Traversal (BFS): ");
    levelOrderTraversal(root);
    printf("\n");

    return 0;
}

```

Binary Tree Utility Functions

```

#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

typedef struct Node {
    int data;
    struct Node* left;
    struct Node* right;
} Node;

Node* createNode(int data) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = data;
}

```

```

        newNode->left = newNode->right = NULL;
        return newNode;
    }

    // Calculate height of tree
    int height(Node* root) {
        if (root == NULL) return -1; // Height of empty tree is -1

        int leftHeight = height(root->left);
        int rightHeight = height(root->right);

        return 1 + (leftHeight > rightHeight ? leftHeight : rightHeight);
    }

    // Count total nodes
    int countNodes(Node* root) {
        if (root == NULL) return 0;
        return 1 + countNodes(root->left) + countNodes(root->right);
    }

    // Count leaf nodes
    int countLeaves(Node* root) {
        if (root == NULL) return 0;
        if (root->left == NULL && root->right == NULL) return 1;
        return countLeaves(root->left) + countLeaves(root->right);
    }

    // Find maximum value
    int findMax(Node* root) {
        if (root == NULL) return -1;

        int max = root->data;
        int leftMax = findMax(root->left);
        int rightMax = findMax(root->right);

        if (leftMax > max) max = leftMax;
        if (rightMax > max) max = rightMax;

        return max;
    }

    // Search for a value
    bool search(Node* root, int target) {
        if (root == NULL) return false;
        if (root->data == target) return true;

        return search(root->left, target) || search(root->right, target);
    }

    // Check if tree is balanced
    bool isBalanced(Node* root) {
        if (root == NULL) return true;

        int leftHeight = height(root->left);
        int rightHeight = height(root->right);
    }

```

```

        if (abs(leftHeight - rightHeight) > 1) return false;

        return isBalanced(root->left) && isBalanced(root->right);
    }

// Free entire tree
void freeTree(Node* root) {
    if (root == NULL) return;

    freeTree(root->left);
    freeTree(root->right);
    free(root);
}

int main() {
    // Create tree
    Node* root = createNode(1);
    root->left = createNode(2);
    root->right = createNode(3);
    root->left->left = createNode(4);
    root->left->right = createNode(5);

    printf("=== Binary Tree Utility Functions ===\n\n");

    printf("Height of tree: %d\n", height(root));
    printf("Total nodes: %d\n", countNodes(root));
    printf("Leaf nodes: %d\n", countLeaves(root));
    printf("Maximum value: %d\n", findMax(root));
    printf("Search for 5: %s\n", search(root, 5) ? "Found" : "Not Found");
    printf("Is balanced: %s\n", isBalanced(root) ? "Yes" : "No");

    freeTree(root);
    return 0;
}

```

Binary Search Tree Implementation in C

BST Node Structure and Basic Operations

```

#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

typedef struct Node {
    int data;
    struct Node* left;
    struct Node* right;
} Node;

// Create new node
Node* createNode(int data) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = data;

```

```

        newNode->left = newNode->right = NULL;
        return newNode;
    }

// Insert node into BST (Recursive)
Node* insert(Node* root, int data) {
    // Base case: empty tree or reached leaf
    if (root == NULL) {
        return createNode(data);
    }

    // Recursive case: navigate to correct position
    if (data < root->data) {
        root->left = insert(root->left, data);
    } else if (data > root->data) {
        root->right = insert(root->right, data);
    }
    // If data == root->data, don't insert (no duplicates)

    return root;
}

// Search for value in BST (Recursive)
bool search(Node* root, int target) {
    // Base cases
    if (root == NULL) return false;
    if (root->data == target) return true;

    // Recursive navigation
    if (target < root->data) {
        return search(root->left, target);
    } else {
        return search(root->right, target);
    }
}

// Search (Iterative) - More efficient
bool searchIterative(Node* root, int target) {
    Node* current = root;

    while (current != NULL) {
        if (current->data == target) return true;
        if (target < current->data) {
            current = current->left;
        } else {
            current = current->right;
        }
    }

    return false;
}

// Find minimum value (leftmost node)
Node* findMin(Node* root) {
    if (root == NULL) return NULL;

```

```

        while (root->left != NULL) {
            root = root->left;
        }
        return root;
    }

// Find maximum value (rightmost node)
Node* findMax(Node* root) {
    if (root == NULL) return NULL;

    while (root->right != NULL) {
        root = root->right;
    }
    return root;
}

// Inorder traversal (gives sorted order)
void inorder(Node* root) {
    if (root == NULL) return;
    inorder(root->left);
    printf("%d ", root->data);
    inorder(root->right);
}

int main() {
    printf("=== Binary Search Tree Operations ===\n\n");

    Node* root = NULL;

    // Insert values
    printf("Inserting: 50, 30, 70, 20, 40, 60, 80\n");
    root = insert(root, 50);
    root = insert(root, 30);
    root = insert(root, 70);
    root = insert(root, 20);
    root = insert(root, 40);
    root = insert(root, 60);
    root = insert(root, 80);

    /*
    * Created BST:
    *           50
    *          /  \
    *         30   70
    *        / \   / \
    *       20 40 60 80
    */

    printf("\nInorder Traversal (sorted): ");
    inorder(root);
    printf("\n\n");

    // Search operations
    int searchValues[] = {40, 100};
    for (int i = 0; i < 2; i++) {
        printf("Search %d: %s\n", searchValues[i],

```

```

        search(root, searchValues[i]) ? "Found" : "Not Found");
    }

    // Find min/max
    Node* minNode = findMin(root);
    Node* maxNode = findMax(root);
    printf("\nMinimum value: %d\n", minNode->data);
    printf("Maximum value: %d\n", maxNode->data);

    return 0;
}

```

BST Deletion (Advanced Operation)

```

#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node* left;
    struct Node* right;
} Node;

Node* createNode(int data) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = data;
    newNode->left = newNode->right = NULL;
    return newNode;
}

Node* insert(Node* root, int data) {
    if (root == NULL) return createNode(data);

    if (data < root->data)
        root->left = insert(root->left, data);
    else if (data > root->data)
        root->right = insert(root->right, data);

    return root;
}

Node* findMin(Node* root) {
    while (root->left != NULL) root = root->left;
    return root;
}

// Delete node from BST
Node* deleteNode(Node* root, int target) {
    // Base case: empty tree
    if (root == NULL) return NULL;

    // Navigate to node to delete
    if (target < root->data) {
        root->left = deleteNode(root->left, target);
    } else if (target > root->data) {

```

```

        root->right = deleteNode(root->right, target);
    } else {
        // Found node to delete

        // Case 1: Leaf node (no children)
        if (root->left == NULL && root->right == NULL) {
            free(root);
            return NULL;
        }

        // Case 2: One child
        else if (root->left == NULL) {
            Node* temp = root->right;
            free(root);
            return temp;
        } else if (root->right == NULL) {
            Node* temp = root->left;
            free(root);
            return temp;
        }

        // Case 3: Two children
        else {
            // Find inorder successor (minimum in right subtree)
            Node* successor = findMin(root->right);

            // Copy successor's data to current node
            root->data = successor->data;

            // Delete successor
            root->right = deleteNode(root->right, successor->data);
        }
    }

    return root;
}

void inorder(Node* root) {
    if (root == NULL) return;
    inorder(root->left);
    printf("%d ", root->data);
    inorder(root->right);
}

int main() {
    printf("=== BST Deletion Demo ===\n\n");

    Node* root = NULL;
    int values[] = {50, 30, 70, 20, 40, 60, 80, 10, 25, 65};

    printf("Inserting values: ");
    for (int i = 0; i < 10; i++) {
        root = insert(root, values[i]);
        printf("%d ", values[i]);
    }
}

```

```

printf("\n\nInitial tree (inorder): ");
inorder(root);

// Test deletions
printf("\n\nDeleting leaf node (10):\n");
root = deleteNode(root, 10);
printf("After deletion: ");
inorder(root);

printf("\n\nDeleting node with one child (20):\n");
root = deleteNode(root, 20);
printf("After deletion: ");
inorder(root);

printf("\n\nDeleting node with two children (30):\n");
root = deleteNode(root, 30);
printf("After deletion: ");
inorder(root);

printf("\n\nDeleting root (50):\n");
root = deleteNode(root, 50);
printf("After deletion: ");
inorder(root);
printf("\n");

return 0;
}

```

Advanced BST Operations

```

#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <limits.h>

typedef struct Node {
    int data;
    struct Node* left;
    struct Node* right;
} Node;

Node* createNode(int data) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = data;
    newNode->left = newNode->right = NULL;
    return newNode;
}

// Validate if tree is a valid BST
bool isBSTUtil(Node* root, int min, int max) {
    if (root == NULL) return true;

    // Current node must be in range [min, max]
    if (root->data <= min || root->data >= max)
        return false;
}

```

```

        // Check left subtree with updated max
        // Check right subtree with updated min
        return isBSTUtil(root->left, min, root->data) &&
               isBSTUtil(root->right, root->data, max);
    }

    bool isBST(Node* root) {
        return isBSTUtil(root, INT_MIN, INT_MAX);
    }

    // Find kth smallest element (1-indexed)
    void kthSmallestUtil(Node* root, int k, int* count, int* result) {
        if (root == NULL || *count >= k) return;

        kthSmallestUtil(root->left, k, count, result);

        (*count)++;
        if (*count == k) {
            *result = root->data;
            return;
        }

        kthSmallestUtil(root->right, k, count, result);
    }

    int kthSmallest(Node* root, int k) {
        int count = 0;
        int result = -1;
        kthSmallestUtil(root, k, &count, &result);
        return result;
    }

    // Find Lowest Common Ancestor (LCA)
    Node* findLCA(Node* root, int n1, int n2) {
        if (root == NULL) return NULL;

        // If both n1 and n2 are smaller, LCA is in left subtree
        if (root->data > n1 && root->data > n2)
            return findLCA(root->left, n1, n2);

        // If both n1 and n2 are larger, LCA is in right subtree
        if (root->data < n1 && root->data < n2)
            return findLCA(root->right, n1, n2);

        // We're at the split point - this is the LCA
        return root;
    }

    // Count nodes in range [low, high]
    int countInRange(Node* root, int low, int high) {
        if (root == NULL) return 0;

        // If current node is in range
        if (root->data >= low && root->data <= high) {
            return 1 + countInRange(root->left, low, high) +

```

```

        countInRange(root->right, low, high);
    }

    // If current < low, only check right subtree
    if (root->data < low) {
        return countInRange(root->right, low, high);
    }

    // If current > high, only check left subtree
    return countInRange(root->left, low, high);
}

// Print nodes in range [low, high]
void printRange(Node* root, int low, int high) {
    if (root == NULL) return;

    // If current node is in range, print it
    if (root->data >= low && root->data <= high) {
        printRange(root->left, low, high);
        printf("%d ", root->data);
        printRange(root->right, low, high);
    }

    // Navigate based on BST property
    else if (root->data < low) {
        printRange(root->right, low, high);
    } else {
        printRange(root->left, low, high);
    }
}

Node* insert(Node* root, int data) {
    if (root == NULL) return createNode(data);
    if (data < root->data)
        root->left = insert(root->left, data);
    else if (data > root->data)
        root->right = insert(root->right, data);
    return root;
}

void inorder(Node* root) {
    if (root == NULL) return;
    inorder(root->left);
    printf("%d ", root->data);
    inorder(root->right);
}

int main() {
    printf("=== Advanced BST Operations ===\n\n");

    Node* root = NULL;
    int values[] = {50, 30, 70, 20, 40, 60, 80, 10, 25, 35, 65};

    for (int i = 0; i < 11; i++) {
        root = insert(root, values[i]);
    }
}

```

```

    printf("BST (inorder): ");
    inorder(root);
    printf("\n\n");

    // Validate BST
    printf("Is valid BST: %s\n\n", isBST(root) ? "Yes" : "No");

    // Kth smallest
    int k = 3;
    printf("%dth smallest element: %d\n\n", k, kthSmallest(root, k));

    // Lowest Common Ancestor
    int n1 = 10, n2 = 35;
    Node* lca = findLCA(root, n1, n2);
    printf("LCA of %d and %d: %d\n\n", n1, n2, lca->data);

    // Range operations
    int low = 25, high = 65;
    printf("Nodes in range [%d, %d]: ", low, high);
    printRange(root, low, high);
    printf("\n");
    printf("Count in range: %d\n", countInRange(root, low, high));

    return 0;
}

```

Summary and Best Practices

When to Use Binary Trees vs BST

Use Binary Trees when:

- Representing hierarchical relationships (family trees, org charts)
- Implementing heaps (complete binary trees)
- Expression parsing (expression trees)
- Decision trees
- Huffman coding

Use BST when:

- Need fast search, insertion, deletion ($O(\log n)$ average)
- Maintaining sorted data dynamically
- Range queries
- Finding predecessors/successors
- Database indexing

Time Complexity Summary

Binary Search Tree Operations:

Operation	Average Case	Worst Case (Skewed)	Best Case
Search	$O(\log n)$	$O(n)$	$O(1)$
Insert	$O(\log n)$	$O(n)$	$O(1)$
Delete	$O(\log n)$	$O(n)$	$O(1)$
Min/Max	$O(\log n)$	$O(n)$	$O(1)$
Inorder	$O(n)$	$O(n)$	$O(n)$

Note: Self-balancing BSTs (AVL, Red-Black) guarantee $O(\log n)$ worst case.

Space Complexity

- **Storage:** $O(n)$ for n nodes
- **Recursion Stack:** $O(h)$ where h = height
 - Balanced tree: $O(\log n)$
 - Skewed tree: $O(n)$

This comprehensive guide provides the foundation for understanding trees and prepares you for advanced topics like AVL trees, Red-Black trees, B-trees, and tree-based algorithms in your data structures journey.

[1] [2] [3] [4] [5] [6] [7] [8] [9] [10] [11] [12] [13] [14] [15] [16] [17] [18] [19] [20]



1. http://www.btechsmartclass.com/data_structures/binary-tree.html
2. <https://www.programiz.com/dsa/binary-tree>
3. <https://www.programiz.com/dsa/binary-search-tree>
4. <https://www.programiz.com/dsa/tree-traversal>
5. <https://www.geeksforgeeks.org/dsa/introduction-to-binary-tree/>
6. <https://www.c-sharpcorner.com/blogs/basic-binary-search-tree-bst-implementation-without-recursion>
7. <https://www.geeksforgeeks.org/dsa/tree-traversals-inorder-preorder-and-postorder/>
8. <https://algotcademy.com/blog/mastering-binary-trees-for-beginners-a-comprehensive-guide-to-understanding-and-implementing-data-structures/>
9. <https://www.geeksforgeeks.org/dsa/binary-search-tree-data-structure/>
10. <https://builtin.com/software-engineering-perspectives/tree-traversal>
11. <https://www.geeksforgeeks.org/dsa/binary-tree-data-structure/>
12. <https://www.freecodecamp.org/news/binary-search-trees-bst-explained-with-examples/>
13. <https://www.digitalocean.com/community/tutorials/binary-search-tree-bst-search-insert-remove>
14. <https://www.geeksforgeeks.org/dsa/preorder-vs-inorder-vs-postorder/>
15. <https://www.simplilearn.com/tutorials/data-structure-tutorial/binary-tree-in-data-structures>

16. https://www.tutorialspoint.com/data_structures_algorithms/binary_search_tree.htm
17. <https://stackoverflow.com/questions/9456937/when-to-use-preorder-postorder-and-inorder-binary-search-tree-traversal-strategy>
18. https://www.w3schools.com/dsa/dsa_data_binarytrees.php
19. https://en.wikipedia.org/wiki/Binary_Search_Tree
20. <https://www.freecodecamp.org/news/binary-search-tree-traversal-inorder-preorder-post-order-for-bst/>